

Azure AI Lab 9 — Multi-Agent Demand Planner

Running the programs, tracing them in Azure Service Bus, and interpreting the Chaos Drill

FORGE FDE Academy · Batch-1 (Azure track) · Lab 9-A / 9-B + Chaos Drill

Prepared from a live run on the Techademy Azure lab subscription (“Learner Account”). All console output and portal screenshots in this document are from an actual end-to-end run.

1 What this lab contains and how the pieces run

Lab 9 builds a Consumer-Packaged-Goods (CPG) demand planner as an orchestrator – worker – verifier multi-agent system. Two “worker” agents (a sales-baseline analyst and a promo modeller) feed an orchestrator that reconciles a forecast; a verifier / critic agent flags bad forecasts before they are published. The agents are decoupled through Azure Service Bus events, so each can be scaled out independently.

There are four runnable entry points. Each one has a header comment that states exactly how to run it — offline (no Azure) and, where relevant, against real Azure. The table below is the run matrix used for this document.

Program	Lab	Offline command	Azure command
01_run_crew.py	9-A	python 01_run_crew.py --mock	python 01_run_crew.py
02_service_bus_eventing.py	9-B	python 02_service_bus_eventing.py -- local	python 02_service_bus_eventing.py --role produce / --role consume (offline model; uses token prices)
03_cost_per_decision.py	—	python 03_cost_per_decision.py	
04_chaos_drill.py	Drill	python 04_chaos_drill.py --all	(in-process; see §5 for Azure observability)

The offline paths use an in-memory bus and a deterministic pipeline, so the orchestration logic (events, verifier, rework loop, cost) can be exercised without an LLM. The Azure paths use the real CrewAI crew on Azure OpenAI and the real Azure Service Bus queue. The same event names and the same verifier logic run on both paths.

2 Azure resources used in this run

These are the concrete resources the lab .env points at (secrets omitted). You will navigate to the first two in the Azure portal throughout this document.

Resource	Name / value	Where
Service Bus namespace	agent-events-vrl (Basic tier)	East US
Service Bus queue	agent-events-vrl-queue	in the namespace above
Application Insights	appi-lab8 (ikey 399d14e9...)	East US
Azure OpenAI	azuser7553-mmlllocal-8300 · deployment gpt-4.1-mini	used by Lab 9-A crew
Resource group	rg-azuser7553_mmll.local-hWDeY	—

Subscription	Learner Account (4116346b-...)	—
--------------	--------------------------------	---

3 Lab 9-A — run the multi-agent planner (01_run_crew.py)

Header-comment combinations:

```
python 01_run_crew.py --mock      # offline deterministic pipeline (no Azure)
python 01_run_crew.py            # real CrewAI crew on Azure OpenAI (needs .env)
```

3.1 Offline (--mock)

The deterministic pipeline runs the full orchestrator–worker–verifier flow and prints the plan, the four events it emitted, the verifier verdict, MAPE (forecast accuracy vs actuals) and the cost per decision:

```
[bus] using in-memory LocalBus (offline).

----- DEMAND PLAN (mock pipeline) -----
final status : forecast.approved
reworked      : False
MAPE vs actuals: 5.3 %
cost/decision : $ 0.0275
events        : ['baseline.created', 'uplift.created', 'forecast.created',
'forecast.approved']

verifier report:
{
  "passed": true,
  "flags": []
}

forecast:
{
  "SKU-BEV-01": 18931,
  "SKU-BEV-02": 9923,
  "SKU-SNK-01": 13903,
  "SKU-SNK-02": 7489,
  "SKU-HOM-01": 6764,
  "SKU-HOM-02": 4369
}
```

How to read it: the verifier passed (no flags), the plan needed no rework, forecast error (MAPE) is 5.3 %, and one decision costs \$0.0275. The four events — baseline.created → uplift.created → forecast.created → forecast.approved — are the messages that, on the Azure path, travel over Service Bus (\$4).

3.2 Real CrewAI crew on Azure OpenAI

With the .env AOAI credentials present, the same steps run as four real LLM agents (Sales Analyst, Promotion Modeler, Demand Planner, Verifier Critic) in sequence. The Verifier Critic independently decided the reconciled forecast was implausible and returned FAIL — exactly the critic behaviour the pattern is designed to give:

----- CREW RESULT -----

FAIL

Flagged SKUs and reasons:

- SKU-BEV-01: Quantity 18931 seems excessively high compared to SKU-BEV-02 at 9917. Unless there is a known capacity increase or promotional uplift, this may indicate over-capacity or an implausible spike.
- SKU-SNK-01: Quantity 13910 is almost double SKU-SNK-02 at 7489. This large discrepancy requires justification; otherwise, it may be an implausible spike.
- SKU-HOM-01: Quantity 6769 is significantly higher than SKU-HOM-02 at 4369. Without promo or uplift explanation, this difference may be questionable.

No information on promotions or uplift is provided, so it is not possible to verify if these high values are justified. Please review the forecast for potential over-capacity or implausible spikes and provide promo context if applicable.

----- TOKEN USAGE (for cost-per-decision) -----

```
{
  "total_tokens": 11412,
  "prompt_tokens": 8584,
  "cached_prompt_tokens": 0,
  "completion_tokens": 2828,
  "reasoning_tokens": 0,
  "cache_creation_tokens": 0,
  "successful_requests": 28
}
```

Trace Batch Finalization

Interpretation: the crew reached a FAIL verdict on its own reasoning (not a hard-coded rule), flagging the three SKUs whose quantities looked like unexplained spikes. Token usage (11,412 tokens over 28 requests) is what feeds the cost-per-decision model in §3.3.

Note on tracing: CREWAI_TRACING_ENABLED=true sends the crew trace to CrewAI's own platform (app.crewai.com), NOT to Application Insights — the run printed a app.crewai.com/.../ephemeral_trace_batches/... link. This lab's code does not wire an Azure Monitor exporter, which is why Application Insights stays empty (see §5.3).

3.3 Cost per decision (03_cost_per_decision.py)

This answers the CRO's two questions — “give me a cost-per-decision number” and “token costs jumped 4x, bring them down without losing quality.” It compares an all-premium configuration against a routed one (cheap models for routine workers, premium only for the orchestrator) and proves quality held via MAPE:

```
=====
COST PER DECISION (routed config): $0.0275
=====
per-agent breakdown: {
  "sales_agent": 0.0009,
  "promo_agent": 0.0008,
  "orchestrator": 0.0245,
  "verifier": 0.0012
}
```

```
}

----- 'Costs jumped 4x' – mitigation by model routing -----
all-premium per decision : $0.0737
routed      per decision : $0.0275
saving      : 62.8%
monthly (20,000 decisions):
  all-premium : $1,475.00
  routed      : $549.10
  saved       : $925.90

----- quality check (did routing hurt accuracy?) -----
forecast MAPE vs actuals : 5.3% (lower is better)
verifier passed          : True
```

Conclusion: routing cut cost-per-decision materially while the verifier still passed and MAPE stayed low – cost down WITHOUT losing quality.

Interpretation: routing cuts cost per decision by 62.8 % (\$0.0737 → \$0.0275) while the verifier still passes and MAPE stays at 5.3 % — cost down, quality intact.

4 Lab 9-B — inter-agent eventing over Azure Service Bus (02_service_bus_eventing.py)

This is the decoupling step: the planner PUBLISHES events as it works, and a separate consumer READS them and reacts (e.g. a notifier pod, or a scaled-out worker). Header-comment combinations:

```
python 02_service_bus_eventing.py --local      # in-memory bus (offline)
python 02_service_bus_eventing.py             # real Azure Service Bus (needs .env)
# With Azure, run a PRODUCER in one terminal and a CONSUMER in another:
python 02_service_bus_eventing.py --role produce      # terminal 1
python 02_service_bus_eventing.py --role consume     # terminal 2
```

The event contract

Event type	Meaning	Consumer reaction
baseline.created	sales worker produced a baseline	log for audit
uplift.created	promo worker produced an uplift	log for audit
forecast.created	orchestrator reconciled a forecast	log for audit
forecast.flagged	verifier rejected a forecast	alert planners
rework.requested	orchestrator asks a worker to redo	re-run the affected worker
forecast.approved	forecast passed — publishable	publish the plan to the S&OP system

4.1 Producer run (terminal 1)

The producer runs one demand-planning decision and publishes its events to the real queue. Because the consumer is NOT run here, the messages stay on the queue:

```
[bus] using Azure Service Bus.
[producer] running a demand-planning decision and publishing events...
[producer] published events: ['baseline.created', 'uplift.created', 'forecast.created',
'forecast.approved']
[producer] final status: forecast.approved

Note: with real Azure Service Bus, the producer and consumer can run in separate
processes/pods and the queue carries events between them.
---- queue runtime after producer ----
active_message_count: 4  total: 4
seq# 17  subject= baseline.created  body= {"type": "baseline.created", "payload":
{"skus": 6}}
seq# 18  subject= uplift.created  body= {"type": "uplift.created", "payload": {"skus":
6}}
seq# 19  subject= forecast.created  body= {"type": "forecast.created", "payload": {"n":
6}}
seq# 20  subject= forecast.approved  body= {"type": "forecast.approved", "payload":
{"passed": true}}
```

The four events are now sitting on agent-events-vrl-queue, each carrying a JSON body and a subject equal to the event type. This is the exact data a downstream pod would receive.

4.2 Where to look in the Azure portal — Service Bus

Portal → search “Service Bus” (or All services → Integration → Service Bus). You land on the namespace list:

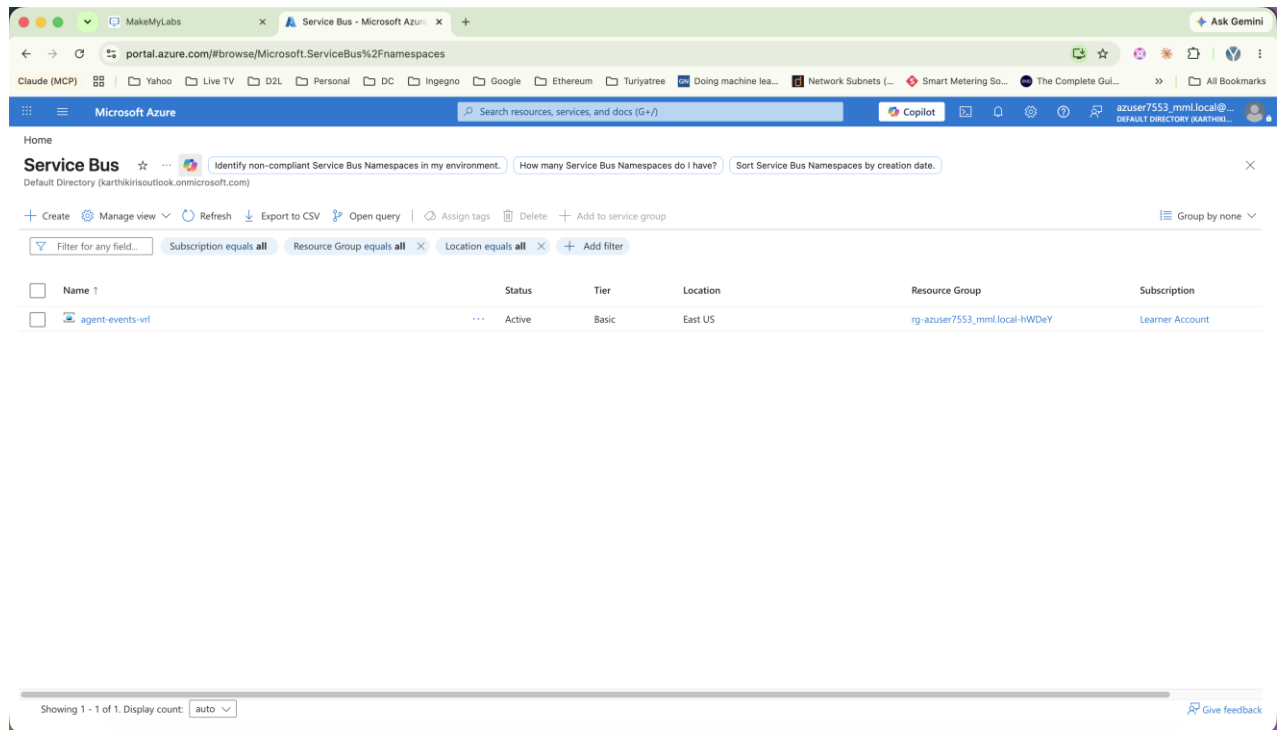


Figure 1 — Service Bus namespace list. The lab namespace `agent-events-vrl` (Basic, East US) in resource group `rg-azuser7553_mml.local-hWDeY`.

Open the namespace. The Overview blade’s Messages chart is the fastest way to see the exchange. Right after the producer run it shows Incoming Messages = 4 and Outgoing Messages = 0 — four events went IN, nothing has been read OUT yet:

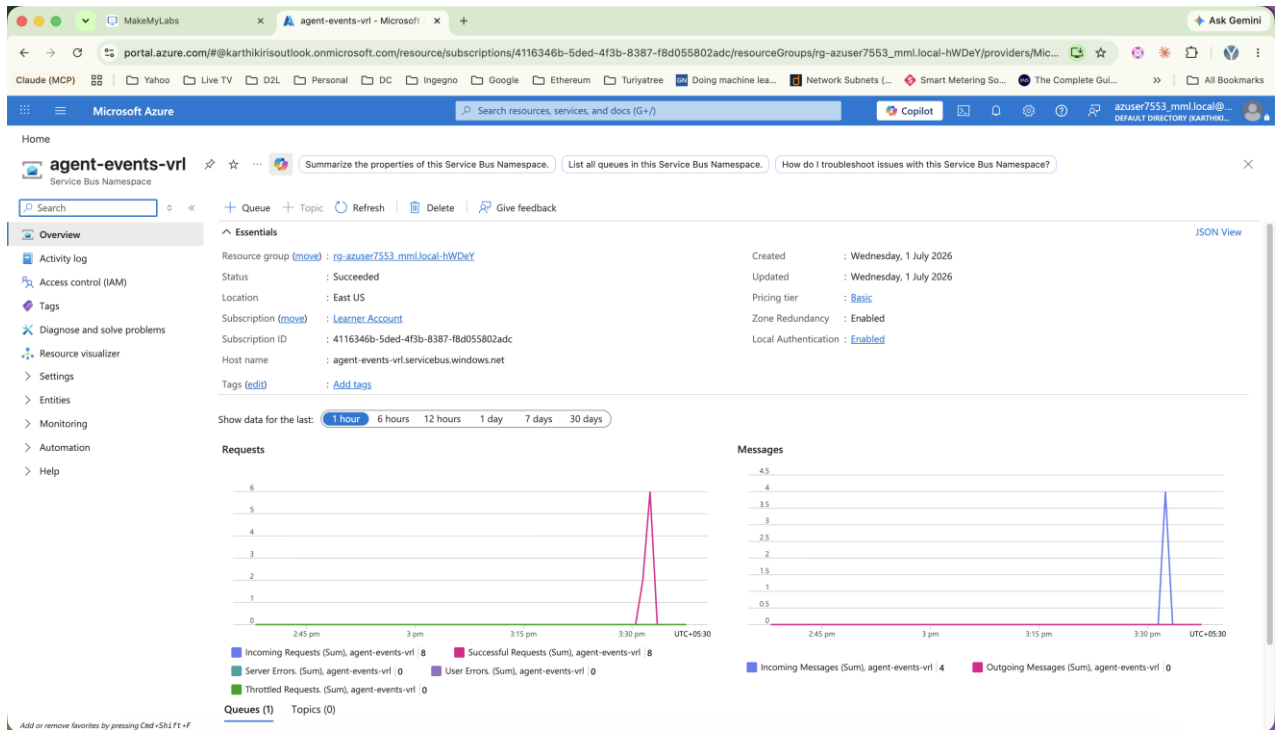


Figure 2 — Namespace Overview after the producer run. Messages chart: Incoming (Sum) = 4, Outgoing (Sum) = 0. Host name agent-events-vrl.servicebus.windows.net.

Drill into the queue: Overview shows Message Counts → Active = 4 messages. These are the producer's events, durably held on the queue (message time-to-live is 14 days), waiting for a consumer:

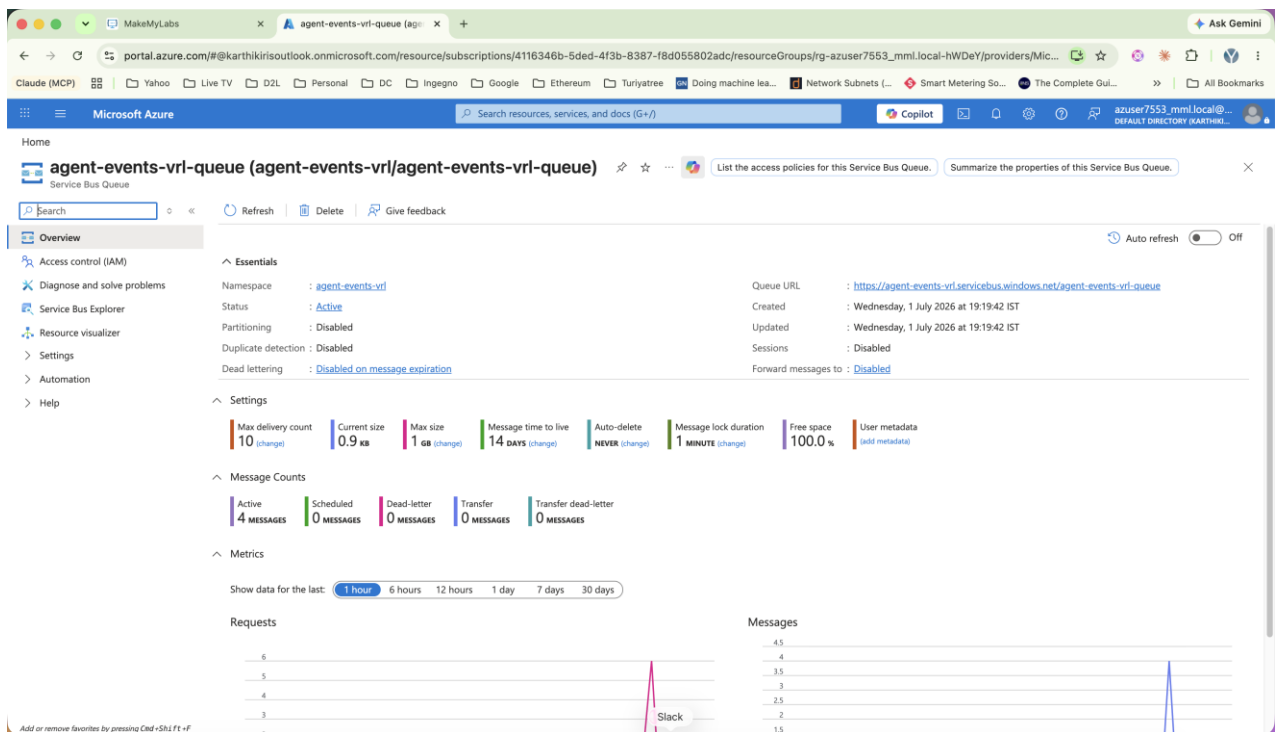


Figure 3 — Queue agent-events-vr1-queue Overview: Active = 4 messages waiting. Note the Service Bus Explorer item in the left menu.

4.3 Reading the actual messages (peek)

To see the message bodies without removing them, use Service Bus Explorer: Queue → Service Bus Explorer (left menu) → receive mode Peek → Peek from start. Each row shows the Sequence #, the Subject (= event type) and the JSON body. The same peek, run from code against the real queue, returns:

```
QUEUE: agent-events-vr1-queue  active_message_count: 4
  seq# 17  subject= baseline.created  body= {"type": "baseline.created", "payload":
{"skus": 6}}
  seq# 18  subject= uplift.created    body= {"type": "uplift.created",   "payload":
{"skus": 6}}
  seq# 19  subject= forecast.created  body= {"type": "forecast.created", "payload": {"n":
6}}
  seq# 20  subject= forecast.approved body= {"type": "forecast.approved", "payload":
{"passed": true}}
```

Peek is non-destructive (the messages remain Active). A consumer that calls receive instead will complete (delete) each message as it processes it.

4.4 Consumer run (terminal 2) and the completed exchange

Now run the consumer. It reads the four events off the real queue and reacts per the event contract, then the queue drains to zero:

```
[bus] using Azure Service Bus.

[consumer] reading events off the bus and reacting:
  event baseline.created  -> log for audit
  event uplift.created    -> log for audit
  event forecast.created  -> log for audit
  event forecast.approved -> publish the plan to the S&OP system

Note: with real Azure Service Bus, the producer and consumer can run in separate
processes/pods and the queue carries events between them.
---- queue runtime AFTER consumer ----
active_message_count: 0  total: 0
```

Back on the namespace Overview (give metrics ~1–2 minutes to aggregate, then Refresh) the Messages chart now shows BOTH Incoming = 4 and Outgoing = 4 — the producer put four events in, the consumer took four out. That is the data exchange, end to end, across two separate processes:

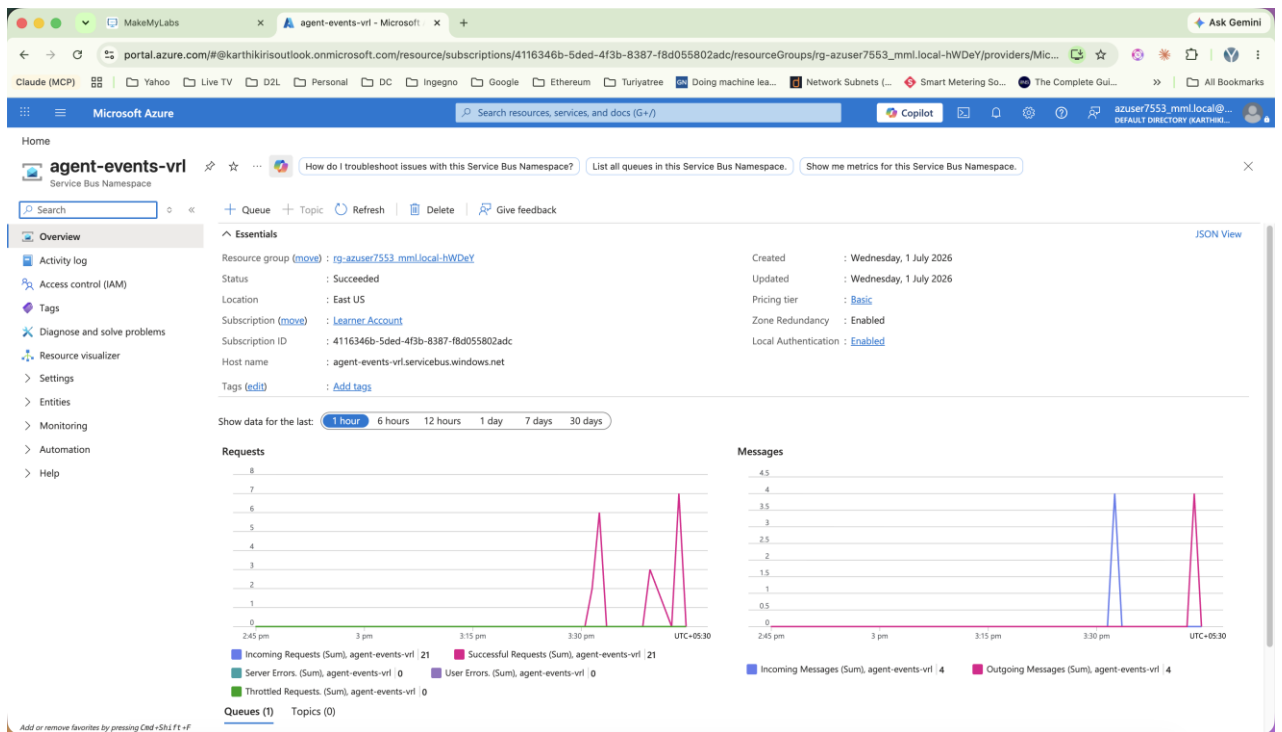


Figure 4 — Namespace Overview after the consumer run. Messages chart: Incoming (Sum) = 4 (blue, producer) AND Outgoing (Sum) = 4 (pink, consumer). The queue is empty again.

How to trace a run:

- **Namespace → Overview** the Messages chart (Incoming vs Outgoing) tells you whether a producer, a consumer, or both ran, and how many events moved.
- **Queue → Overview** Active message count tells you the backlog (how many events are waiting for a consumer right now).
- **Queue → Service Bus Explorer** Peek shows the event bodies/subjects without consuming them — use it to confirm the exact payload a downstream agent will get.
- **Namespace → Overview** Requests chart (Incoming/Successful/Server-Errors/Throttled) shows call health; a spike in Server Errors or Throttled is your first failure signal.
- **Monitoring** Monitoring → Metrics lets you pin these to a dashboard or alert (e.g. alert when Active messages > N for M minutes — a stuck consumer).

5 The Chaos Drill — inject faults, read the scorecard (04_chaos_drill.py)

The drill is the mentor’s control panel for the live 30-minute chaos test: it injects one or more faults into a demand-planning decision and prints a scorecard on four dimensions — did the system stay up, did the verifier catch the bad forecast, did cost stay controlled, and was the failure observable. Header-comment combinations:

```
python 04_chaos_drill.py --faults bad_sales_data
python 04_chaos_drill.py --faults sales_timeout,cost_spike
python 04_chaos_drill.py --all          # run each fault scenario in turn
# Fault kinds: sales_timeout, bad_sales_data, cost_spike, verifier_bypass
```

5.1 The full drill (--all)

```
===== DRILL: baseline (no faults) (faults: none) =====
score: 4/4 [up=True verifier=True cost=True observable=True]
- cost-per-decision $0.0275 (budget $0.05).
- 4 events emitted to the bus.

===== DRILL: sales_timeout (faults: sales_timeout) =====
score: 4/4 [up=True verifier=True cost=True observable=True]
- sales_agent timed out; pod retried with fallback (no fault).
- cost-per-decision $0.0275 (budget $0.05).
- 4 events emitted to the bus.

===== DRILL: bad_sales_data (faults: bad_sales_data) =====
score: 4/4 [up=True verifier=True cost=True observable=True]
- Verifier flagged the corrupted forecast.
- cost-per-decision $0.0275 (budget $0.05).
- 6 events emitted to the bus.

===== DRILL: cost_spike (faults: cost_spike) =====
score: 3/4 [up=True verifier=True cost=False observable=True]
- Model upgraded to all-premium mid-run (cost spike injected).
- cost-per-decision $0.0737 (budget $0.05).
- ACTION: route workers to cheaper models to restore budget.
- 4 events emitted to the bus.

===== DRILL: verifier_bypass (faults: verifier_bypass) =====
score: 4/4 [up=True verifier=True cost=True observable=True]
- cost-per-decision $0.0275 (budget $0.05).
- 4 events emitted to the bus.
- verifier_bypass attempted – verifier still ran (cannot be skipped).

===== DRILL: compound (timeout + cost spike) (faults: sales_timeout, cost_spike) =====
score: 3/4 [up=True verifier=True cost=False observable=True]
- Model upgraded to all-premium mid-run (cost spike injected).
- sales_agent timed out; pod retried with fallback (no fault).
```

- cost-per-decision \$0.0737 (budget \$0.05).
- ACTION: route workers to cheaper models to restore budget.
- 4 events emitted to the bus.

Drill complete. Discuss: which failures were caught, which needed a human, and what the pod would change to score 4/4 every time.

5.2 Reading the scorecard

The score is out of 4 — one point per dimension. Here is what each fault proves and why the score lands where it does:

Fault (what the mentor breaks)	Score	What happened / how to read it
baseline (no faults)	4/4	Healthy reference run: verifier passes, cost \$0.0275 < \$0.05 budget, 4 events emitted.
sales_timeout	4/4	Sales worker hangs; the pod RETRIES with a fallback (no fault) and stays up. Resilience pattern works.
bad_sales_data	4/4	A corrupted 10-million spike is injected; the VERIFIER CATCHES it → forecast.flagged + rework.requested (6 events). This is the money shot of the critic pattern.
cost_spike	3/4	Model 'upgraded' to all-premium mid-run → cost \$0.0737 > \$0.05 budget. Cost dimension FAILS. Action: route workers back to cheaper models.
verifier_bypass	4/4	Anti-pattern: an attempt to skip the verifier. The verifier always runs, so the bypass cannot succeed — by design.
compound (timeout + cost_spike)	3/4	Stays up (retry) and verifier holds, but the cost spike still blows the budget → 3/4. Compound faults expose the weakest dimension.

Discussion prompt (built into the drill): which failures were caught automatically, which needed a human, and what the pod would change to score 4/4 every time (answer: cap cost with model routing under load, and keep the verifier non-bypassable).

5.3 Where the failure shows up — Service Bus and Application Insights

The drill as shipped runs the pipeline on the in-memory LocalBus, so nothing reaches Azure during --all. To make a failure observable on Azure Service Bus, point the same pipeline at the Azure bus while injecting the fault (the pipeline already accepts both a bus and a fault):

```
from service_bus import get_bus      # AzureBus when SERVICEBUS_CONNECTION_STRING is set
from pipeline import run_once
from chaos import Fault
run_once(bus=get_bus(), fault=Fault(kinds=["bad_sales_data"]))
```

This publishes the fault path — including the failure events — to the real queue. The verifier catches the injected spike (rule V2), the orchestrator requests rework, and the run ends rejected:

```
[bus] using Azure Service Bus.
final_status : forecast.rejected
reworked      : True
events       : ['baseline.created', 'uplift.created', 'forecast.created',
'forecast.flagged', 'rework.requested', 'forecast.rejected']
verifier      : {"passed": false, "flags": [{"sku": "SKU-BEV-01", "rule": "V5", "message":
```

```
"SKU is on promo (25% off) but forecast 13056 is below recent average 13670 \u2014 uplift may be missing."}}}
```

active_message_count on queue: 6

```
seq# 21 subject= baseline.created body= {"type": "baseline.created", "payload": {"skus": 6}}
seq# 22 subject= uplift.created body= {"type": "uplift.created", "payload": {"skus": 6}}
seq# 23 subject= forecast.created body= {"type": "forecast.created", "payload": {"n": 6}}
seq# 24 subject= forecast.flagged body= {"type": "forecast.flagged", "payload": {"flags": [{"sku": "SKU-BEV-01", "rule": "V2", "message": "Forecast 14
seq# 25 subject= rework.requested body= {"type": "rework.requested", "payload": {"skus": 6}}
seq# 26 subject= forecast.rejected body= {"type": "forecast.rejected", "payload": {"passed": false}}
```

On the queue Overview you now see Active = 6 messages — the two extra ones are the failure signals forecast.flagged and rework.requested that a monitoring pod would alert on:

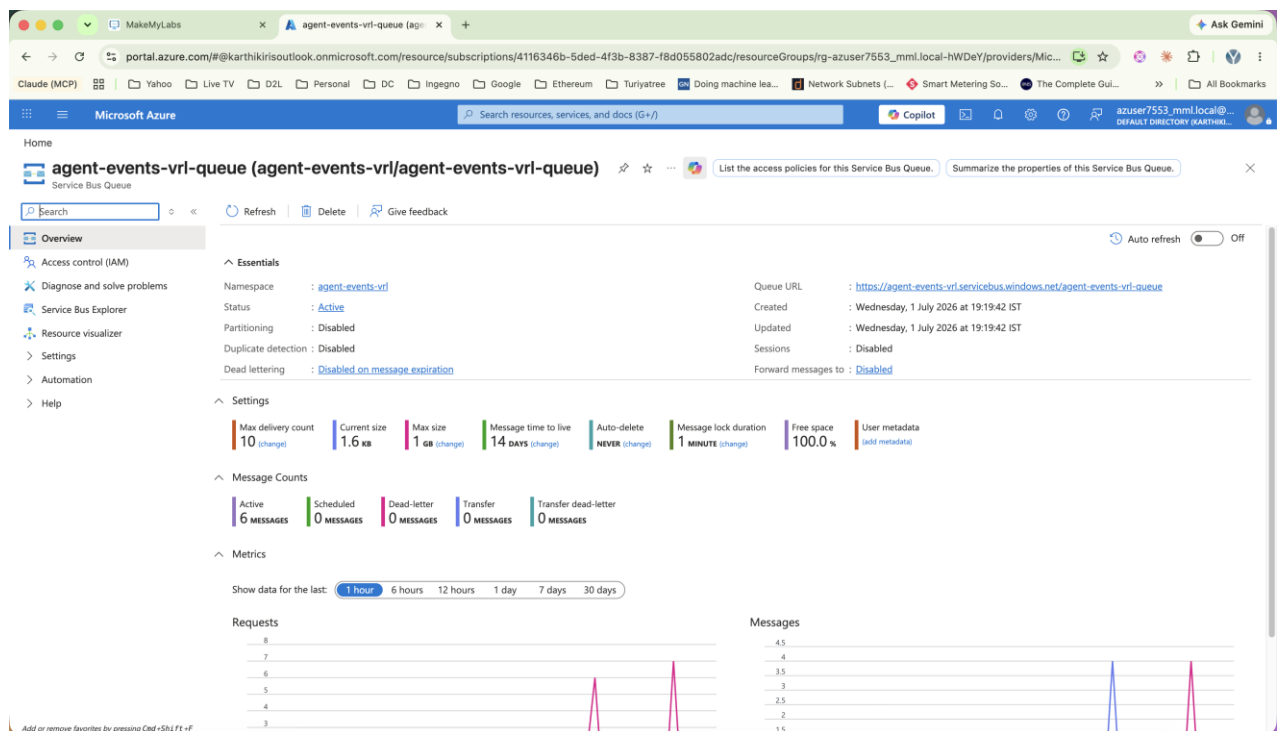


Figure 5 — Queue Overview during the bad_sales_data fault: Active = 6 messages, including forecast.flagged and rework.requested. This is the failure, observable on Service Bus.

Application Insights (appi-lab8).

Application Insights is provisioned for the lab (connection string in .env), and it is where request/dependency/exception telemetry and distributed traces belong. In the portal: App Insights → Investigate → Transaction search / Failures for individual traces and exceptions, or Monitoring → Logs (KQL) to query. In the screenshot below its charts are empty because — at that point — the lab code

attached no OpenTelemetry / Azure Monitor exporter, so the CrewAI trace went to app.crewai.com instead (§3.2):

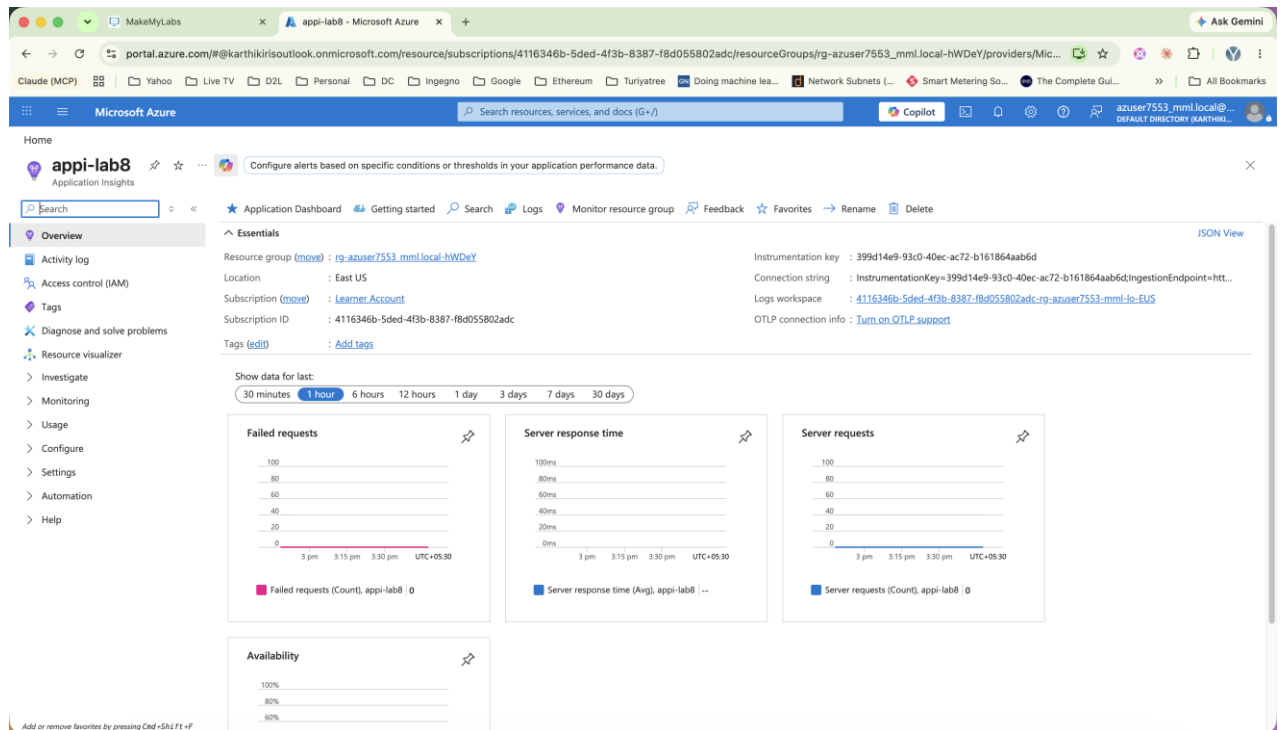


Figure 6 — Application Insights appi-lab8 Overview BEFORE the exporter was wired: Failed requests = 0 / Server requests = 0, because nothing exported yet. §6 wires it and shows the telemetry that then appears.

To make the multi-agent system emit real traces/failures into App Insights you add the exporter once at start-up (it reads APPLICATIONINSIGHTS_CONNECTION_STRING from .env). We implemented exactly this in telemetry.py:

```
pip install azure-monitor-opentelemetry
from azure.monitor.opentelemetry import configure_azure_monitor
configure_azure_monitor() # picks up APPLICATIONINSIGHTS_CONNECTION_STRING
# CrewAI/OpenTelemetry spans + exceptions then flow to appi-lab8
```

Then, in App Insights → Logs, these KQL queries surface the drill's failures:

```
exceptions | where timestamp > ago(1h) | order by timestamp desc // injected faults
traces | where message has "forecast.flagged" or message has "rework" // verifier
catches
requests | where success == false | summarize count() by name // failed
decisions
```

Wiring this exporter next to CrewAI's existing tracing was not free, though — it raised a chain of dependency and provider conflicts. Because learning to diagnose those is the real FDE skill, §6 walks through exactly what broke and how to fix it.

6 Coexisting observability tools — the conflicts to look for (and fix)

Lab 9 runs THREE observability paths at once: Azure Service Bus metrics (§4), CrewAI’s own trace to app.crewai.com (§3.2), and Application Insights via OpenTelemetry (§5.3). Bolting the App Insights exporter onto a stack that already traced through CrewAI surfaced a chain of conflicts that are typical whenever you combine tracing / observability tools on one agent. This section is the field guide — what to look for, how to spot it, and how to fix it. This is a core FDE skill: on a client engagement you will almost always be adding your tooling next to something that is already there.

6.1 The conflict we actually hit

CrewAI and the Azure Monitor exporter both ride the SAME OpenTelemetry core (the opentelemetry-api / -sdk / -semantic-conventions packages). They wanted different versions of it:

- **crewai 1.15.1** declares opentelemetry-sdk $\sim=1.42.0$ (i.e. $\geq 1.42.0, < 1.43$).
- **azure-monitor-opentelemetry 1.8.9** (exporter 1.0.0b55) needs opentelemetry-sdk $\geq 1.43.0$.

A single resolve of both therefore fails outright:

```
$ uv pip compile requirements.txt      # crewai + azure-monitor-opentelemetry
x No solution found when resolving dependencies:
  Because crewai==1.15.1 depends on opentelemetry-sdk>=1.42.0,<1.43.dev0
  and azure-monitor-opentelemetry depends on opentelemetry-sdk>=1.43.0,<1.44.dev0,
  we can conclude that your requirements are incompatible.
```

The trap underneath it: the ONE azure-monitor build that fits crewai’s < 1.43 range (1.6.4 / 1.8.2, exporter 1.0.0b45) INSTALLS cleanly but then dies at import — its code calls a symbol the newer SDK moved:

```
ImportError: cannot import name 'LogData' from 'opentelemetry.sdk._logs'
```

So “pip installed successfully” did NOT mean “it works.” The version math was satisfied; the runtime API was not. Always import and run, never trust the installer alone.

6.2 The conflict classes to look for

Nearly every multi-tool observability problem is one of these eight. Learn the symptom and the detection command for each — that is what turns a two-day mystery into a ten-minute fix.

Conflict class	Symptom	How to detect	How to resolve
Shared-core version pin (diamond dependency): two tools pin different versions of one shared telemetry core	Resolver: “No solution found” / ResolutionImpossible	uv pip compile (or pip install --dry-run); read each pin with importlib.metadata.requires('pkg')	Use a tool version inside the other’s range; else install sequentially so the tolerant lib takes the bump; else isolate in a sidecar /

			separate process
Resolves but won't import (stale API pin): version math OK, but code calls a moved/removed symbol	ImportError / AttributeError at import — not at install time	Never trust “Successfully installed”; add an import + one real call as a CI smoke test	Upgrade the exporter / instrumentation to the build that matches the installed core's API
Global provider contention: OpenTelemetry allows ONE TracerProvider / LoggerProvider per process	One tool's spans vanish, or all spans land in the wrong backend; “Overriding of current TracerProvider is not allowed”	Grep for set_tracer_provider / configure_* calls; watch for the override warning	Pick ONE owner of the provider and attach BOTH exporters to it (add a second SpanProcessor); or fix init order
Multiple transports per framework: a tool may emit on more than one channel	Turning one tool 'off' doesn't stop all its data; two backends disagree	Map each tool → its transport (OTel? vendor HTTP SDK? webhook?)	Independent transports coexist freely; only reconcile the ones sharing the OTel pipe
OTel version lockstep: api / sdk / semantic-conventions / instrumentation must all move together	Half-upgraded stack (mixed 1.42/1.43 or 0.63b/0.64b) → subtle AttributeErrors	pip list grep opentelemetry — confirm one coherent generation	Upgrade or downgrade the whole OTel family as a set, never one package alone
Sampling & double-counting: more exporters = more volume; the same span exported twice	“Data is being sampled” banner; inflated counts; missing events	Watch the App Insights sampling banner; compare counts to what you emitted	Configure sampling centrally; ensure each span is exported exactly once
Config / env collisions: tools read overlapping env (OTEL_SERVICE_NAME, OTEL_EXPORTER_*, connection strings)	Telemetry mislabeled or sent to the wrong resource	Audit env; print the effective config at startup	Set each tool's identity explicitly (e.g. a per-role service name)
Cost, egress & data governance: every exporter is another destination seeing your traces (prompts / PII)	Duplicate ingestion bills; prompts leaving to an extra vendor	List every telemetry destination and what payload it carries	Scrub / limit payloads; confirm each destination is authorised

6.3 How we made CrewAI + Azure Monitor coexist

Three moves resolved every item above without breaking CrewAI:

- **Two-step install.** crewai runs fine on opentelemetry-sdk 1.43 even though it PINS ~=1.42, so install the exporter as a SECOND step — each command solves independently and the later one wins the bump. The base requirements.txt stays conflict-free.

```
uv pip install -r requirements.txt          # crewai etc. (otel 1.42)
uv pip install --upgrade azure-monitor-opentelemetry # bumps otel -> 1.43
```

- **Optional + guarded.** telemetry.py is a no-op unless APPLICATIONINSIGHTS_CONNECTION_STRING is set AND the package imports — so --mock / --local / test_offline.py keep running with zero Azure dependencies.
- **Additive, not replacing.** configure_azure_monitor() ADDS an Azure Monitor exporter to the OTEL provider; it does not touch CrewAI's separate app.crewai.com HTTP trace. Both fire from the same run.

Because the exporter is initialised BEFORE the crew runs, CrewAI's own OTEL spans (the LiteLLM chat calls) also flow to App Insights — a free bonus, at no cost to the app.crewai.com trace. Proof, from one real crew run:

```
[telemetry] Application Insights exporter ON (role=lab9-crew).
...
https://app.crewai.com/crewai_plus/ephemeral_trace_batches/af651eb5-...    <- CrewAI trace
STILL fires
```

And the telemetry lands in appi-lab8. Transaction search shows the run's spans — including CrewAI's own chat / Task Execution spans picked up off the shared OTEL provider:

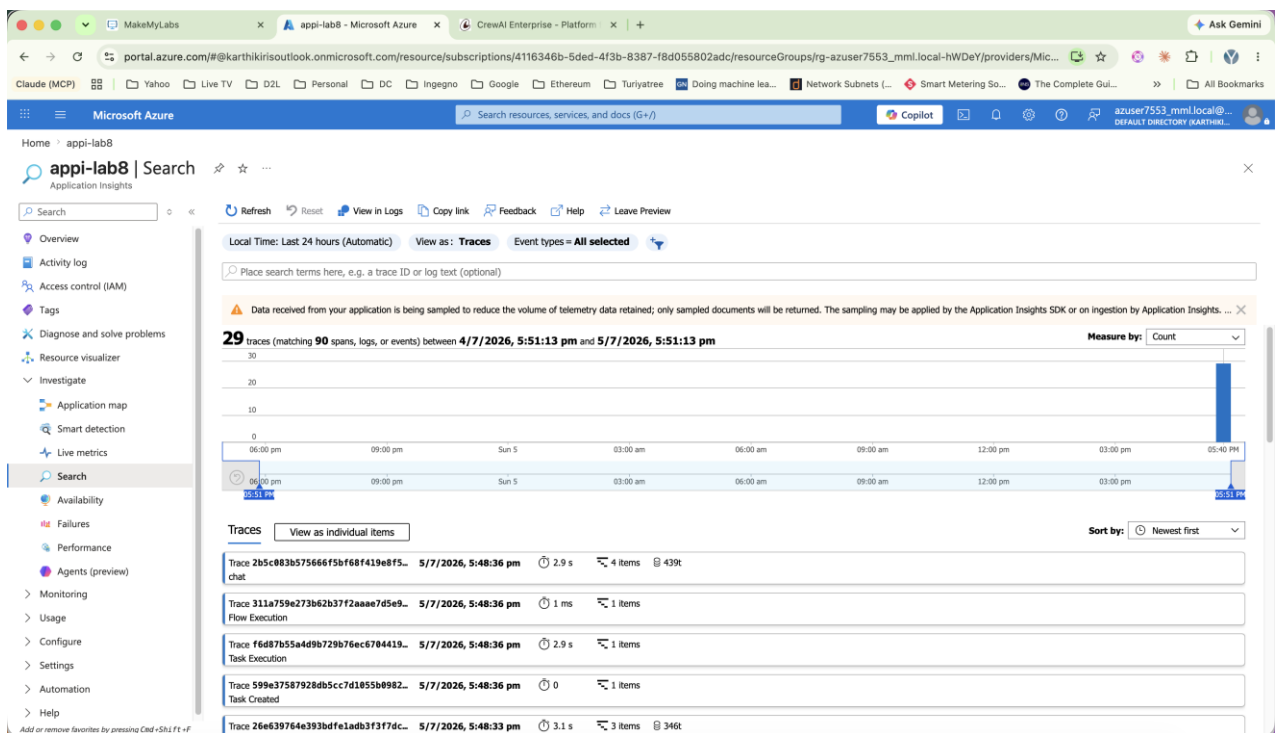


Figure 7 — App Insights → Transaction search: 29 traces / 90 spans-logs-events from the run, including CrewAI's chat and Task Execution spans. (Note the sampling banner at the top — that is the 'sampling & double-counting' conflict class in the wild.)

The injected chaos fault is now a first-class, queryable exception — the failure scenario is observable in App Insights, exactly as it should be:

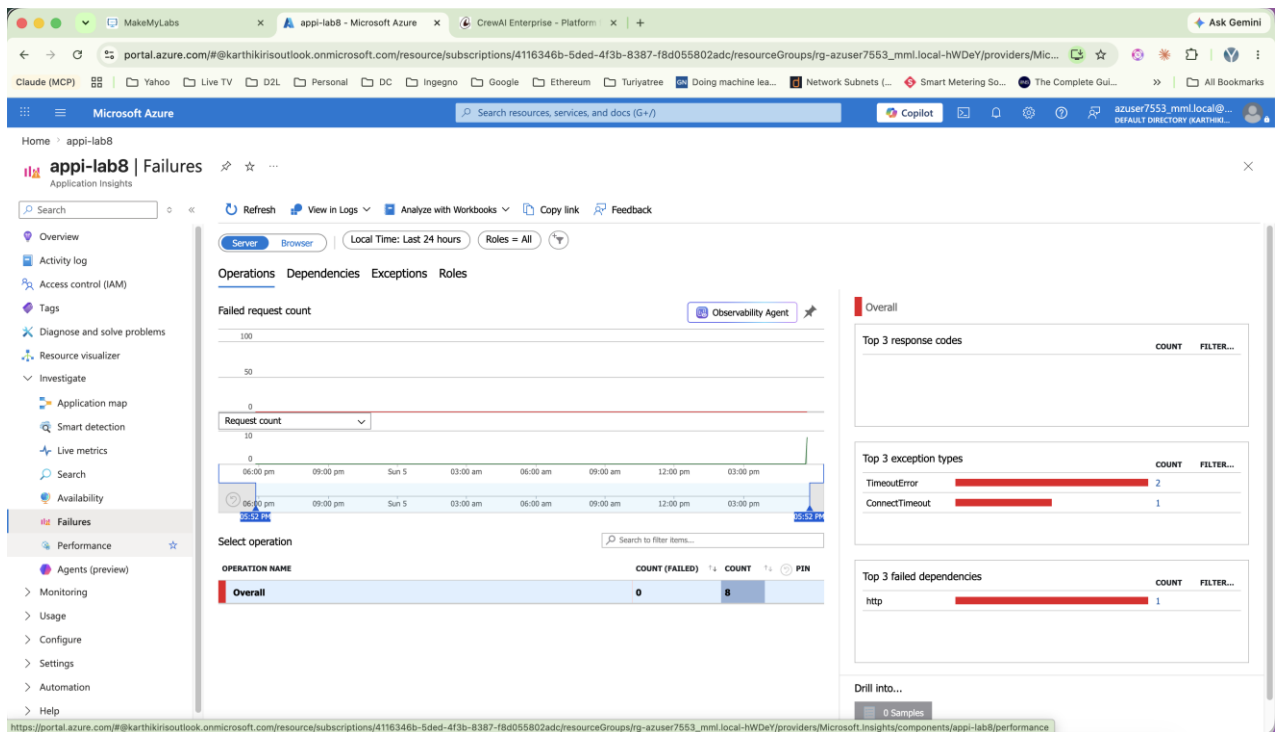


Figure 8 — App Insights → Failures → exception types: TimeoutError x2 — the injected sales_timeout chaos fault, recorded via telemetry.record_exception, next to the pipeline-decision requests.

6.4 Pre-flight checklist before adding any observability tool

- Read its OTEL pin (importlib.metadata.requires) and compare to what is already installed.
- Dry-run the resolve (uv pip compile) BEFORE committing — expect, and read, a “No solution”.
- After install, IMPORT it and run one real call — install success is not runtime success.
- Identify each tool’s transport; only OTEL-sharing tools contend for the global provider.
- Decide who OWNS the TracerProvider; attach extra exporters as processors rather than re-setting it.
- Keep the new exporter OPTIONAL and no-op when unconfigured, so offline paths never depend on it.
- Check sampling and destinations — volume, double-counting, cost, and what data leaves the process.

7 Quick reference

Run everything offline (no Azure)

```
python generate_cpg_data.py          # once, builds the synthetic CPG dataset
python 01_run_crew.py --mock
python 02_service_bus_eventing.py --local
python 03_cost_per_decision.py
python 04_chaos_drill.py --all
```

Run against real Azure (fill in .env first)

```
python 01_run_crew.py                # real CrewAI crew on Azure OpenAI
python 02_service_bus_eventing.py --role produce # terminal 1 (publish)
python 02_service_bus_eventing.py --role consume # terminal 2 (consume)
```

Enable App Insights tracing (optional, one extra step — see §6)

```
uv pip install --upgrade azure-monitor-opentelemetry # bumps OpenTelemetry to 1.43
# then any real-Azure run (and 04_chaos_drill.py) auto-exports to appi-lab8;
# --mock / --local stay telemetry-free by contract.
```

Portal navigation map

I want to see...	Portal path
That events were published/consumed	Service Bus → agent-events-vrl → Overview → Messages chart (Incoming vs Outgoing)
How many events are waiting	... → Entities → Queues → agent-events-vrl-queue → Overview → Message Counts (Active)
The actual event bodies	... queue → Service Bus Explorer → Peek → Peek from start
Call health / throttling / errors	namespace → Overview → Requests chart, or Monitoring → Metrics
Agent traces / LLM spans (App Insights)	Application Insights appi-lab8 → Investigate → Transaction search, or Logs (KQL): traces / requests
Injected fault as an exception	Application Insights appi-lab8 → Investigate → Failures → Exceptions (e.g. TimeoutError), or Logs: exceptions

Generated from a live end-to-end run — console output and portal screenshots are real. Synthetic CPG data is teaching data. Secrets (keys, connection strings) are intentionally omitted.